7-5 · Hybrid Search 구현 · 실습 1.5H

Hybrid 결합과 주차 산출물 완성

RRF 열 줄, 답변 체인, 그리고 산출물 두 가지

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

이 시간의 목표

1.5시간 뒤에는

① hybrid.py 의 rrf() 를 이론 시간의 손 계산 예로 확인한다

② 세 방식을 나란히 놓고 k 를 바꿔 합친 점수를 본다

③ rag.py 로 답하게 하고, 지식베이스 밖 질문의 답을 살핀다

④ vector-schema.yaml · build_kb.py 를 검증해 제출한다

제출 기준 — check_schema.py 통과 · build_kb.py 두 번의 총 개수가 같다 · grep "sk-" 에 아무것도 없다

실습 흐름

아홉 단계 — 결합 · 답변 · 산출물 · 제출

1H 실습의 kb_corpus.py · retrievers.py 를 불러 씁니다 — 모델은 rag_env.py 한 곳

1·2. RRF	3·4. 비교	5·6. 답변	7·8. 산출물	9. 제출
hybrid.py hybrid_try.py	compare3.py rrf_k.py	rag.py rag_source.py	schema.yaml build_kb.py	캡처 1장 grep sk-

1단계

hybrid.py — RRF 를 구현하세요

RRF 한 줄	scores[doc_id] = scores.get(doc_id, 0.0) + 1.0 / (k + rank + 1)
검증	assert rrf([["A", "B", "C"], ["B", "D", "A"]])[0] == "B"
결합	rrf([bm25_rank(query), vector_rank(query)], k=k)[:top_k]
본문	hybrid_search → [text_by_id[i] for i in hybrid_ids(query, top_k)]
실측	RRF 동작 확인

손 계산(B 가 1등)과 코드가 같은지 먼저 — 순서를 바꾸면 틀려도 모른다 · 뒤 단계가 모두 불러 쓰는 부품

2단계

「작업 순서를 관리하는 도구는?」 세 순위

방식	1등	2등	3등
BM25	운영 절	2장	—
벡터	2장	운영 절	분석 절
하이브리드	운영 절	2장	분석 절

양쪽 상위였던 두 조각이 맨 위(1 · 2등을 바꿔 가져 점수가 같다) · 벡터에만 있던 분석 절도 3위로 남는다

3단계

compare3.py — 질문 셋 × 검색 셋

질문	BM25	벡터	하이브리드
「ERR-4021」	수칙 1등	수칙 1등	수칙 1등
「Kafka 파티션」	없음	3장 스트리밍	3장 스트리밍
「일정을 정해 두고 도는 작업」	2장 · 운영 절	2장 · 운영 절	2장 · 운영 절

BM25 가 비어도 하이브리드는 벡터 쪽을 따라갔다 — 늘 그렇지는 않다, 손해 보는 질문을 찾으면 적어 두세요

4단계

rrf_k.py — k 를 바꾸면 합친 점수는

k	상위 세 점수	1등 / 3등
1	0.8333 · 0.8333 · 0.25	3.33배
60	0.0325 · 0.0325 · 0.0159	2.05배
200	0.0099 · 0.0099 · 0.0049	2.01배

k 가 작을수록 각 검색기의 1등이 강하게, 커질수록 고르게 섞인다 — 기본값 60 은 그 사이의 무난한 지점

5단계

rag.py — 찾은 조각만 근거로 답하게

1. **chain = prompt | llm | StrOutputParser()**
   llm 은 rag_env.py 에서 불러온다(OpenAI 또는 Ollama)

2. **system: 컨텍스트만 근거로 · 없으면 모른다고**
   {context} 에 hybrid_search 결과를 --- 로 이어 넣는다

3. **「작업 순서를 관리하는 도구가 뭐야?」**
   실측: 「Airflow가 있습니다.」 — 지식베이스 근거로 맞게

4. **Ollama 는 질문 하나에 1~4분**
   임베딩 모델과 생성 모델이 번갈아 메모리에 올라간다(실측)

실측 모델

llama3.2

(3B)

5단계 · 지식베이스 밖 질문

「김치찌개 끓이는 법」 — 작은 모델은 지어냈다

1. **지시: 컨텍스트에 없으면 모른다고 답해줘**
   system 메시지에 넣어 두었다
2. **결과: 「… 찻집 요리입니다」를 되풀이**
   실측 llama3.2 — 「모른다」 대신 지어냈다
3. **원인: 작은 모델(3B)이 지시를 못 지켰다**
   RAG 를 묶었다고 환각이 저절로 사라지지 않는다
4. **막는 법: 더 큰 모델 · 점수로 끊기**
   검색 점수가 낮으면 LLM 전에 「지식베이스에 없음」(8주차)

> **OpenAI 경로라면 둘째 답이 어떻게 나왔는지 채팅에 적어 주세요**

6단계

rag_source.py — 답 아래에 근거 문서

1. **by_id = {str(r.id): r.payload ...} → hybrid_ids()**
   고른 조각의 payload 를 그대로 가져온다

2. **srcs = sorted({d.get("source", "?") ...})**
   답 아래에 [근거] 로 덧붙인다

3. **「ERR-4021 이 뜨면 무엇부터 확인해?」**
   커넥션 풀 고갈 · 연결을 닫지 않는 코드 · 연결 수(실측)

4. **[근거] .../manual/guide.txt, .../notes/long_notes.txt**
   사람이 근거를 열어 검증한다 — 7-2 메타데이터가 값을 한다

RAG 의 가치

검증할 수 있는 답

7단계 — 주차 산출물 ①

vector-schema.yaml — 내 경로의 차원으로

항목	Ollama 경로	OpenAI 경로
collection	lecture_kb	같음
embedding.model	embeddinggemma	text-embedding-3-small
dim · vector.size	768 · 768	1536 · 1536
distance	Cosine	같음
payload	text · source · chunk_index	같음
chunking	size 300 · overlap 30	같음
id_rule	uuid5(source#chunk_index)	같음

check_schema.py 실측: 정의서와 실제 일치 — 768 Cosine · 어긋나면 AssertionError(예: 실제 1536 vs 정의서 768)

8단계 — 주차 산출물 ②

build_kb.py — 두 번 돌려 같으면 합격

MinIO 의 .txt 를 source 별로 지우고 다시 넣는다 — uuid5 id 라 몇 번 돌려도 같다

1회	2회	CHUNK_SIZE=200	lecture_kb 는	정리(선택)
적재 10 조각 · 총 10	같은 줄이면 합격(실측)	lecture_kb_200 적재 14 · 총 14	그대로 — 다른 컬렉션에	lecture_kb_200 curl -X DELETE

9단계 — 제출

세 가지를 내고, 키를 한 번 더 확인

1. vector-schema.yaml — 내 경로의 값, check_schema.py 통과
2. build_kb.py (+ 불러 쓰는 rag_env.py · minio_env.py)
3. 두 번 실행한 결과가 같다는 캡처 1장 — 8단계의 두 줄
4. grep -rn "sk-" ... . 로 키가 없는지 · .env 는 넣지 않는다

제출물에 키가 섞여 나가는 사고가 실무에서 자주 납니다

7-5 · 실습 1.5H 마무리

다음은 정리·회고 (0.5H)

7주차 닷새를 한 표로 잇고, 자주 겪는 오류를 짚은 뒤 8주차 Airflow 를 준비합니다

- ✓ **7-1~7-5 한 일과 남은 결과물**
- ✓ **체크리스트로 오늘 실습 점검**
- ✓ **id 타입 · 조사 · 컬렉션 이름 · 재색인**
- ✓ **docker compose version — 8주차 준비**

가져갈 것

제출 완료 · 검색 비교 기록